

Project Documentation: E-Commerce Data Lake on AWS (Steps 1–5)

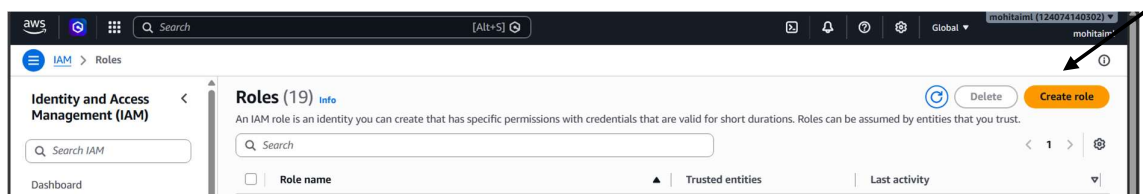
This documentation details the end-to-end implementation of a serverless data lake on AWS, covering storage initialization, cataloging, schema troubleshooting, data cleaning with PySpark, and analytical verification using SQL.

Prerequisites & Identity Management (IAM)

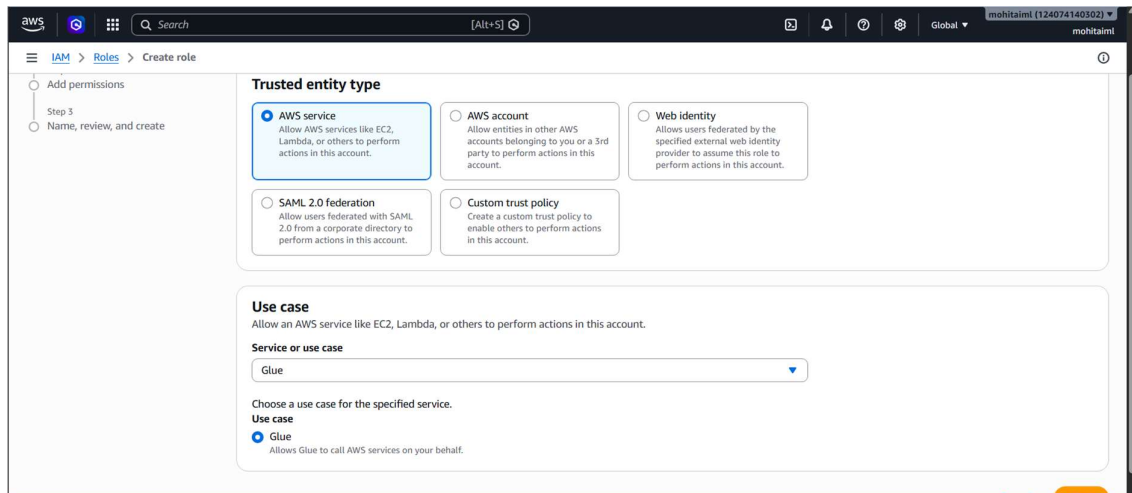
Before building the infrastructure, AWS Glue requires an execution role providing permissions to access S3 data buckets and interact with the Glue Data Catalog.

IAM Configuration Steps

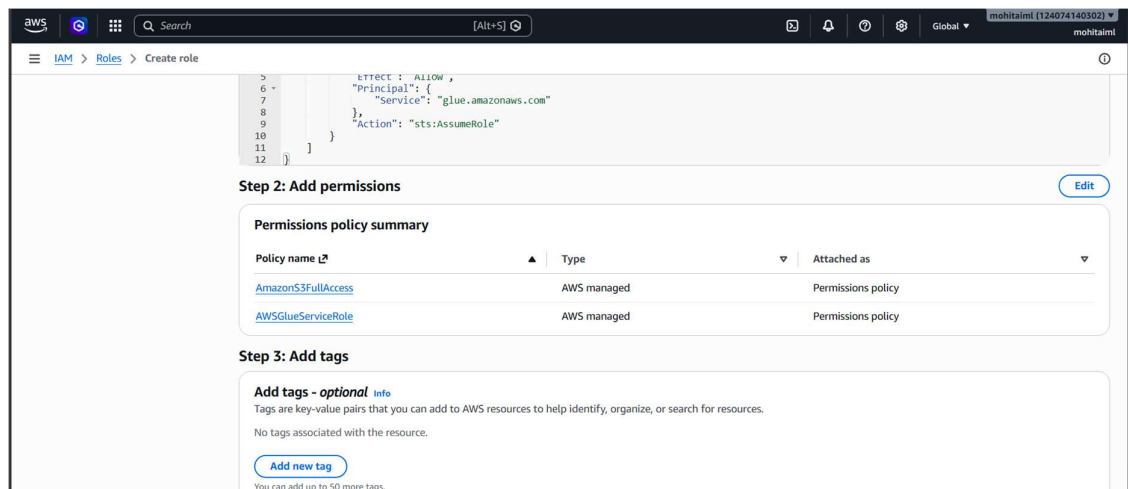
1. Navigate to the **IAM Console > Roles > Create role**.



2. Select **AWS service** as the trusted entity and choose **Glue** from the service dropdown.



3. Attach the following managed policies:
 - AWSGlueServiceRole
 - AmazonS3FullAccess (Note: For production environments, restrict this to specific bucket ARNs using a custom policy).
4. Name the role **GlueDataLakeRole** and complete the creation process.



Step 1: Storage Layer Initialization (Amazon S3)

Amazon S3 acts as the storage backbone, decoupled into a raw immutable zone and an optimized processed zone.

Implementation Steps

1. Navigate to the **S3 Console** and create two buckets using unique identifiers:
 - o yourname-ecommerce-raw-zone
 - o yourname-ecommerce-processed-zone
2. Inside the raw bucket, create three explicit object prefixes (folders):
 - o customers/
 - o products/
 - o orders/
3. Generate the 100-record mock datasets using your local Python generation script and upload each respective CSV file into its matching S3 folder.

Verification Checkpoint (Step 1)

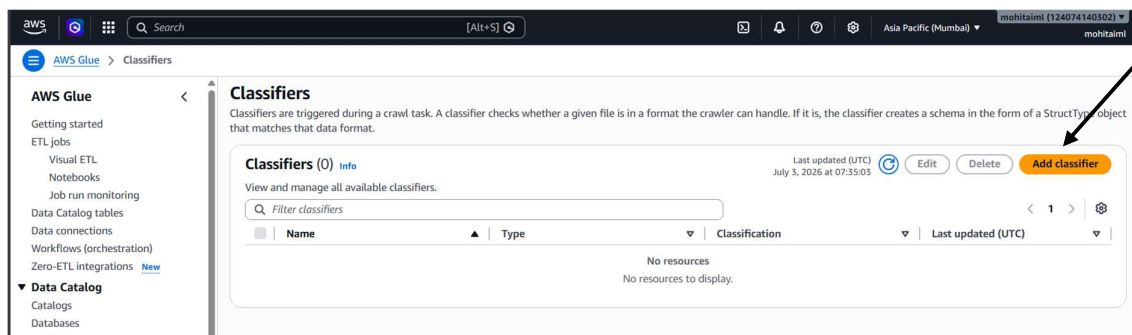
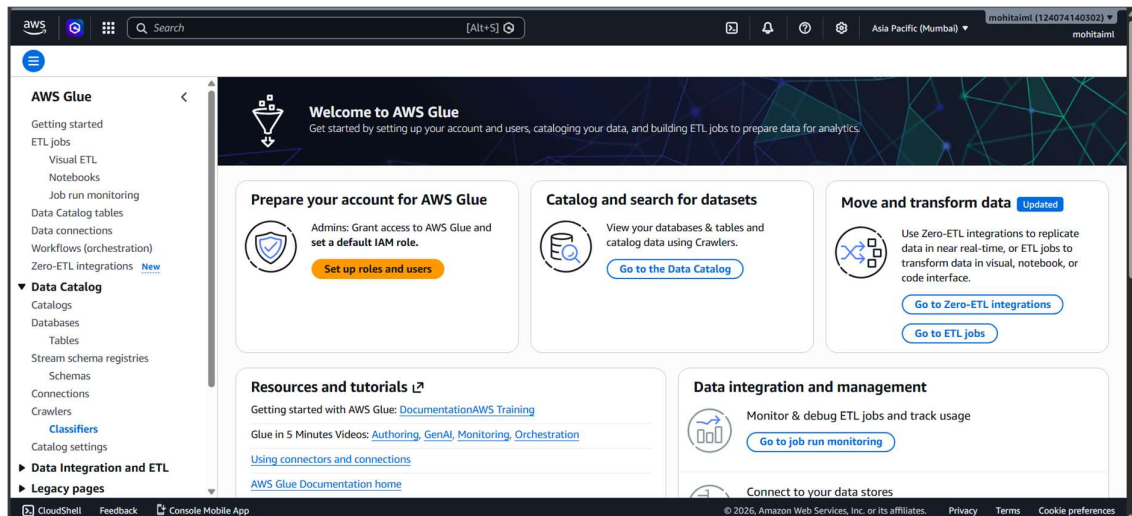
- Open the S3 console and verify that customers.csv, products.csv, and orders.csv reside strictly within their designated subfolders inside the raw bucket.
- Confirm no files are loose in the root directory.

Step 2: Cataloging the Raw Data (AWS Glue Crawler)

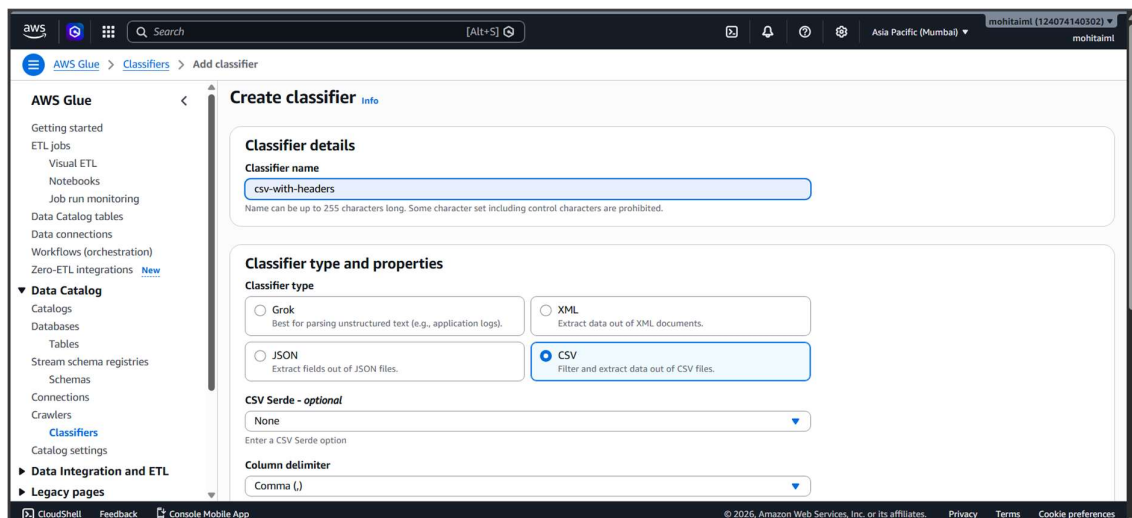
To make unstructured or semi-structured S3 data readable by compute engines, a Glue Crawler scans the files to generate metadata tables.

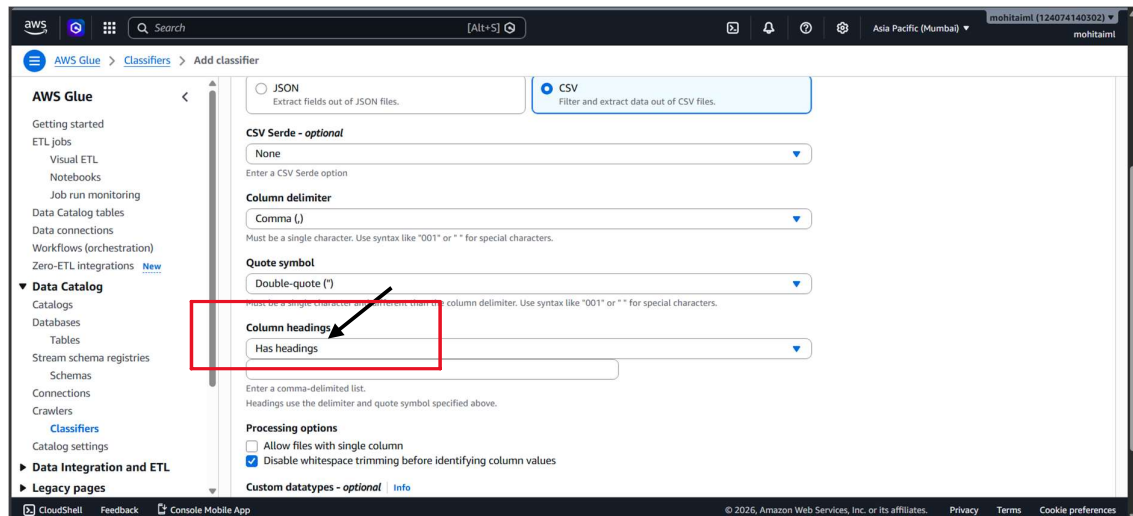
Implementation Steps

1. Navigate to the **AWS Glue Console > Classifiers > Add classifier**.

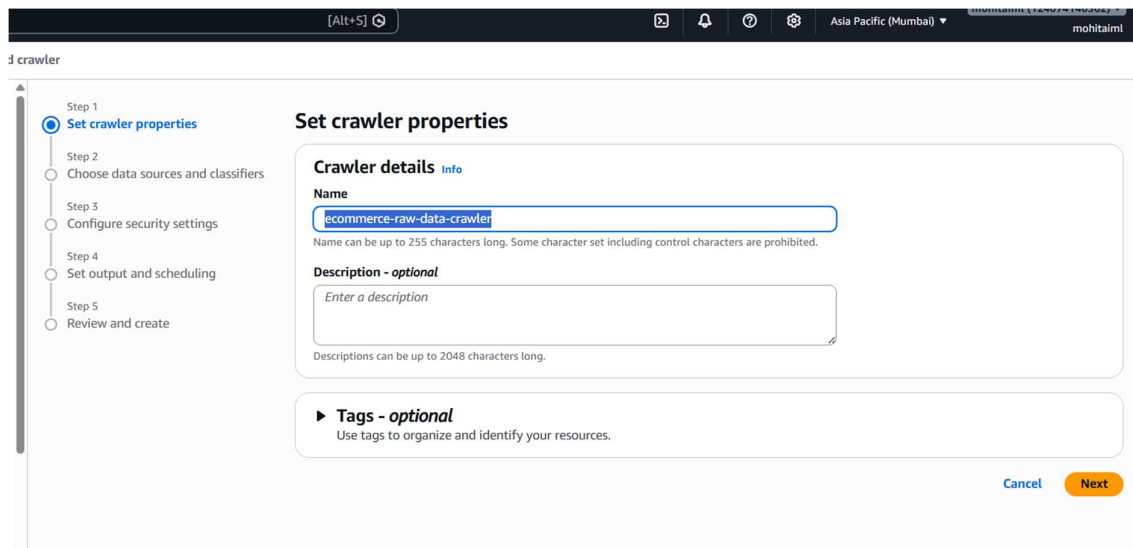
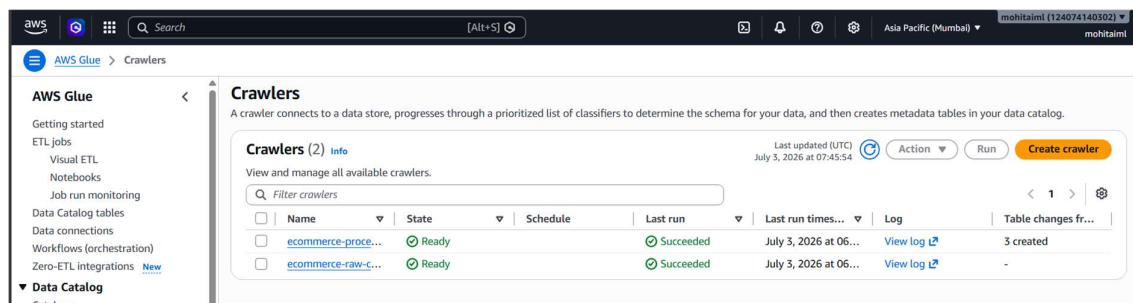


2. Create a CSV classifier named csv-with-headers with **Column headings** set to **Has headings**.

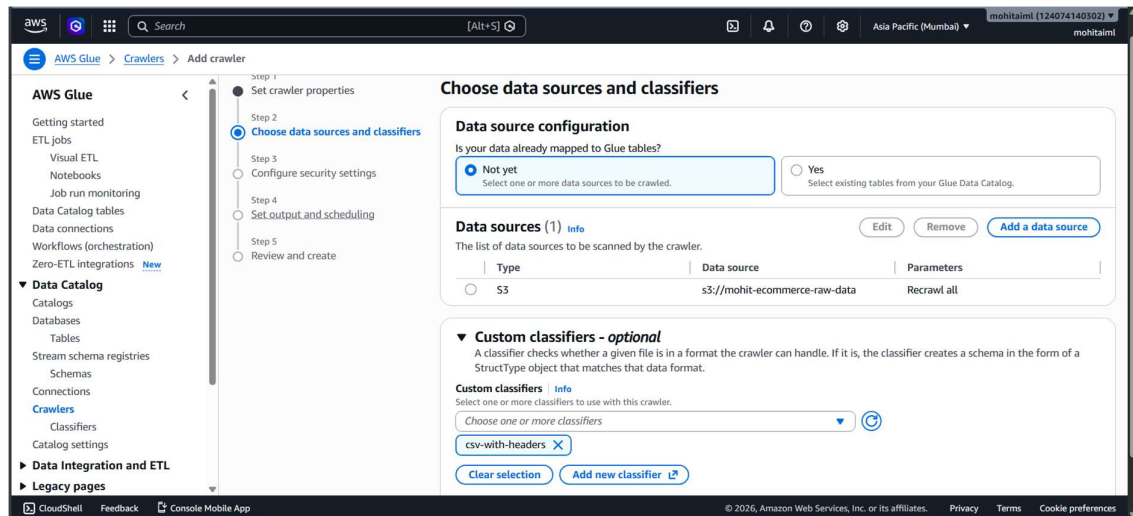




3. Navigate to **Crawlers > Create crawler**. Name it **ecommerce-raw-crawler**.



4. Add the custom classifier **csv-with-headers** to the crawler configuration.



5. Set the data source to the S3 root path of your raw bucket: `s3://yourname-e-commerce-raw-zone/`.

Add data source

Data source

Choose the source of data to be crawled.

S3

Network connection - optional

Optionally include a Network connection to use with this S3 target. Note that each crawler is limited to one Network connection so any other S3 targets will also use the same connection (or none, if left blank).

Clear selection

Add new connection

Location of S3 data

☒ In this account

☐ In a different account

S3 path

Browse for or enter an existing S3 path.

s3://mohit-ecommerce-raw-data

View

Browse S3

All folders and files contained in the S3 path are crawled. For example, type s3://MyBucket/MyFolder/ to crawl all objects in MyFolder within MyBucket.

Subsequent crawler runs

This field is a global field that affects all S3 data sources.

☒ Crawl all sub-folders

Crawl all folders again with every subsequent crawl.

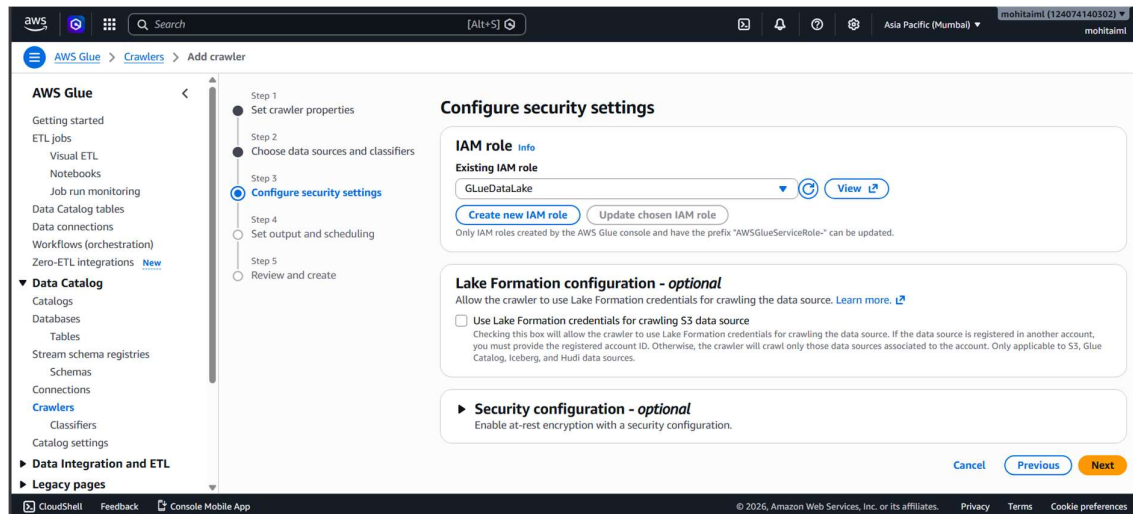
☐ Crawl new sub-folders only

Only Amazon S3 folders that were added since the last crawl will be crawled. If the schemas are compatible, new partitions will be added to existing tables.

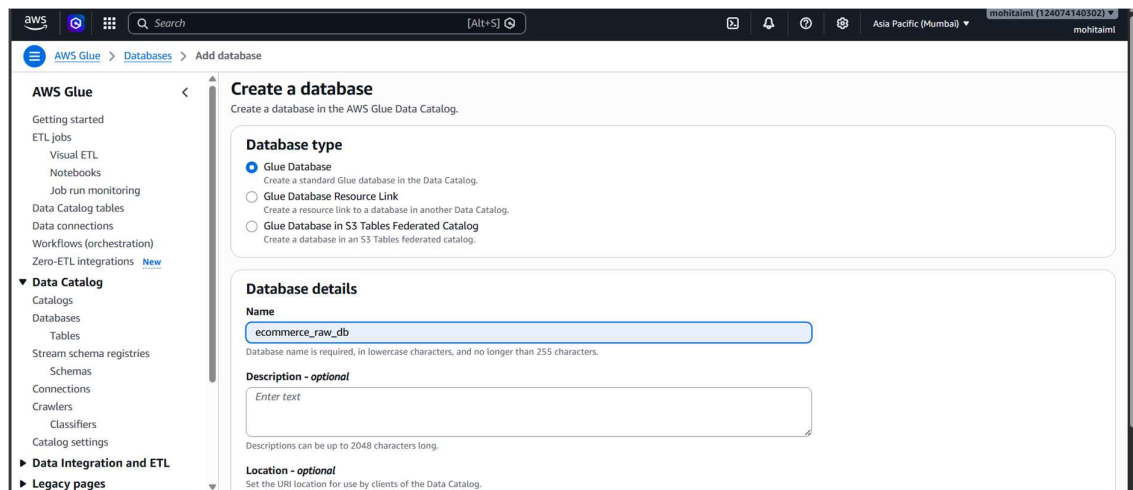
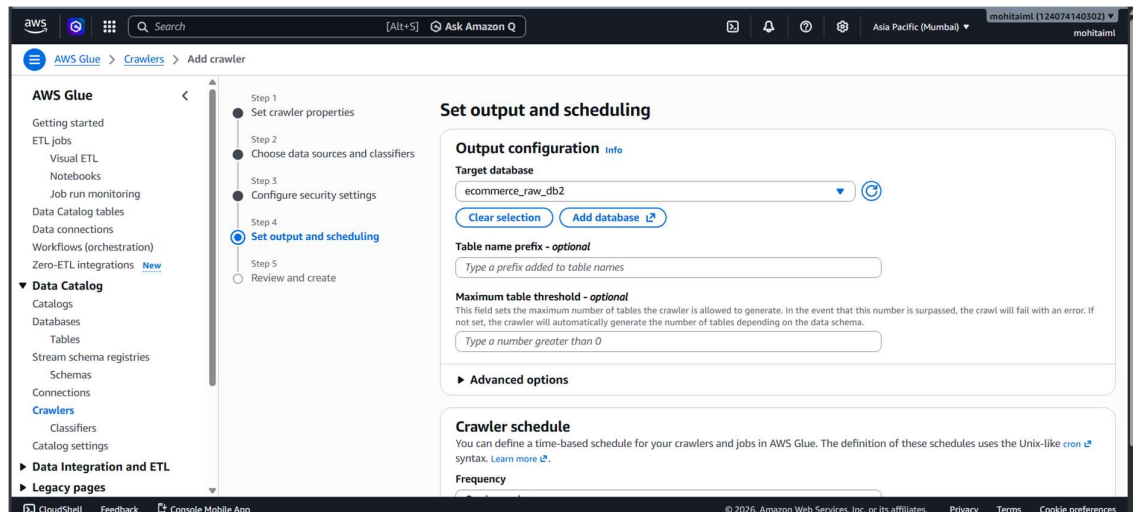
Cancel

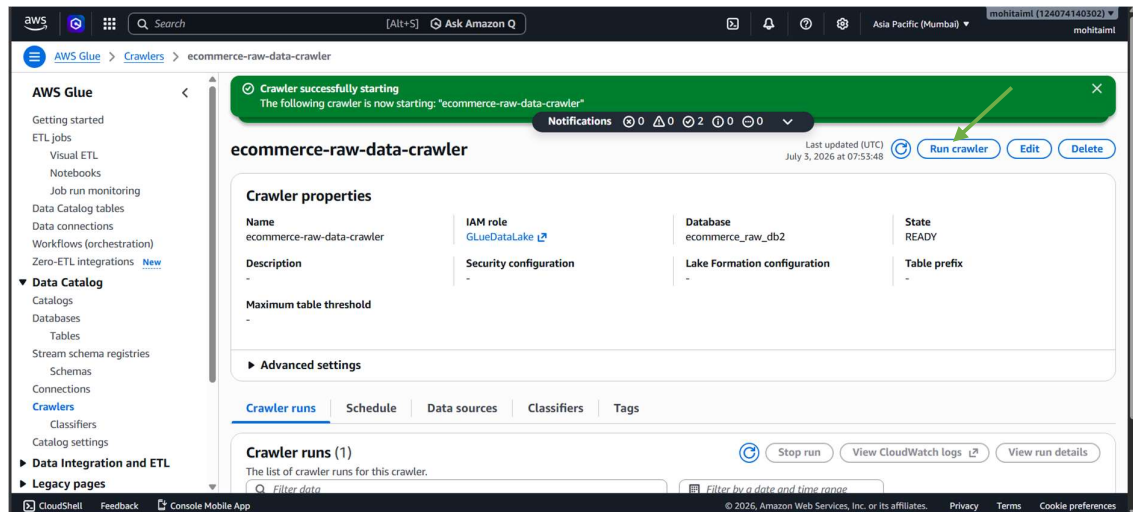
Add an S3 data source

6. Assign the GlueDataLakeRole as the execution role.



7. Create a target database named `ecommerce_raw_db` and map the crawler output to it. Run the crawler.





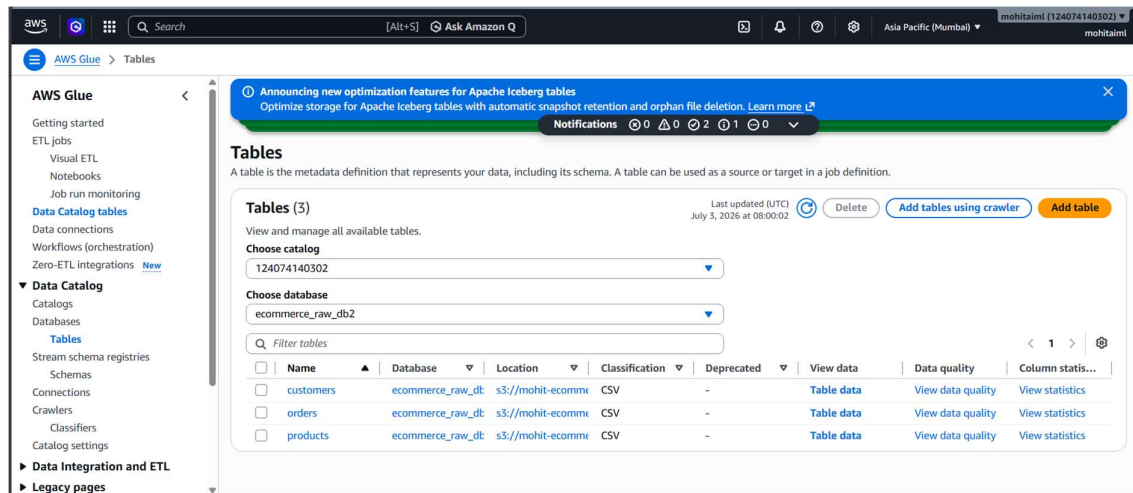
Manual Schema Calibration

If the metadata tables cache an incorrect structure during initial scans, update them manually:

1. Navigate to Glue **Tables** > Select customers.
2. Click **Edit schema** and manually map columns matching your data format:
 - col0 → customer_id
 - col1 → name
 - col2 → email
 - col3 → signup_date

Verification Checkpoint (Step 2)

- Open Glue **Databases** > ecommerce_raw_db > **Tables**.
- Ensure three tables (customers, products, orders) exist with an exact classification type of **csv**.



- Inspect the schemas to ensure clear column identifiers (customer_id, email, etc.) instead of generic labels like col0.

Step 3: Batch Data Transformation (AWS Glue ETL Job)

This step executes a distributed PySpark script to handle typical real-world data anomalies: data type corrections, absolute value normalization for currency anomalies, missing value treatment, and file format conversion to Apache Parquet.

Production PySpark Transformation Script

Python

```
import sys
```

```
from awsglue.transforms import *
```

```
from awsglue.utils import getResolvedOptions
```

```
from pyspark.context import SparkContext
```

```
from awsglue.context import GlueContext
```

```
from awsglue.job import Job
```

```
from pyspark.sql.functions import col, abs
```

```
# Initialize Spark and Glue Contexts
```

```
sc = SparkContext.getOrCreate()
```

```
glueContext = GlueContext(sc)
```

```
spark = glueContext.spark_session
```

```
job = Job(glueContext)
```

```

# =====

# 1. PROCESS CUSTOMERS

# =====

dyf_customers = glueContext.create_dynamic_frame.from_catalog(
    database="ecommerce_raw_db",
    table_name="customers"
)

df_customers = dyf_customers.toDF()

df_customers_clean = df_customers.fillna({"email": "unknown_email@example.com"})

df_customers_clean.write.mode("overwrite").parquet("s3://yourname-ecommerce-processed-zone/customers/")


# =====

# 2. PROCESS PRODUCTS

# =====

dyf_products = glueContext.create_dynamic_frame.from_catalog(
    database="ecommerce_raw_db",
    table_name="products"
)

df_products = dyf_products.toDF()

df_products_clean = df_products.withColumn("price", abs(col("price").cast("float")))\
    .fillna({"category": "Uncategorized"})

df_products_clean.write.mode("overwrite").parquet("s3://yourname-ecommerce-processed-zone/products/")


# =====

# 3. PROCESS ORDERS

# =====

```

```

dyf_orders = glueContext.create_dynamic_frame.from_catalog(
    database="ecommerce_raw_db",
    table_name="orders"
)

df_orders = dyf_orders.toDF()

# Defensive implementation: Safe-cast to string before evaluating nulls or empty spacing
df_orders_clean = df_orders.filter(
    col("quantity").cast("string").isNotNull() &
    (col("quantity").cast("string") != "")
)

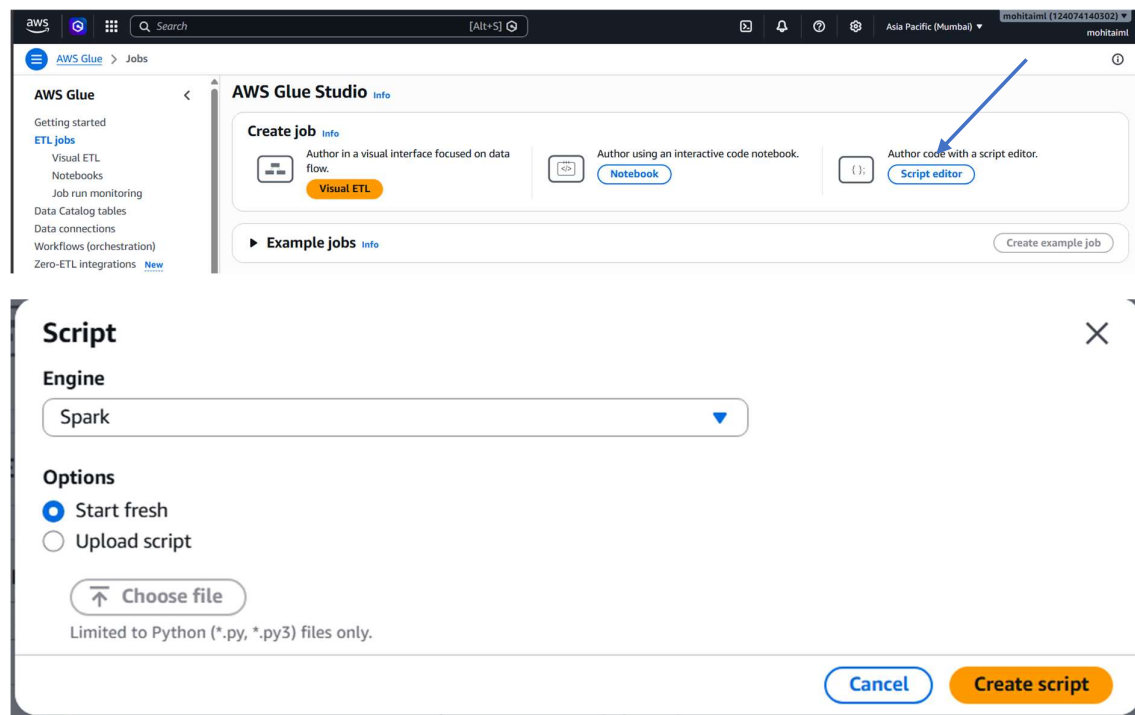
df_orders_clean.write.mode("overwrite").parquet("s3://yourname-ecommerce-processed-
zone/orders/")

job.commit()

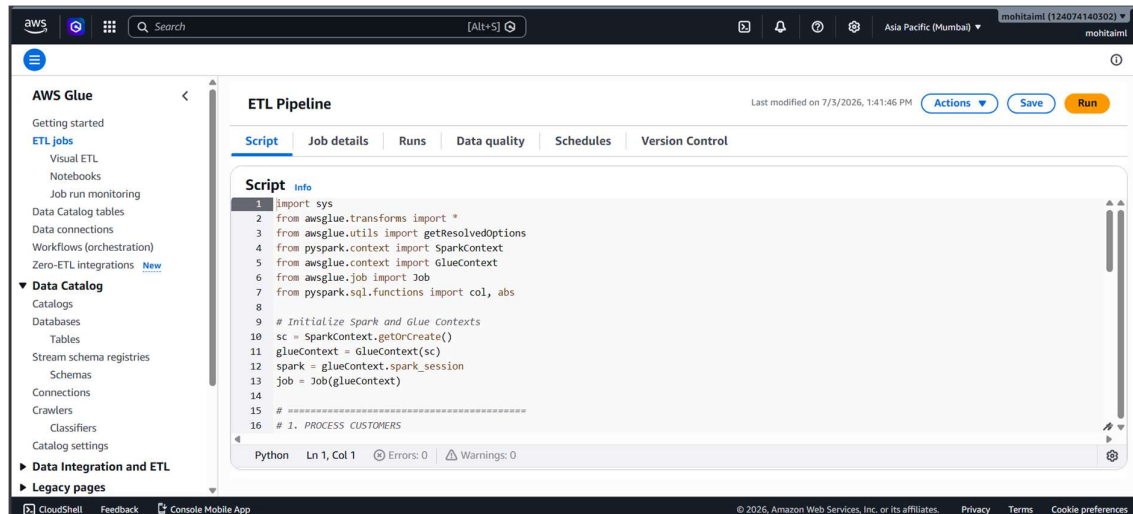
```

Execution Steps

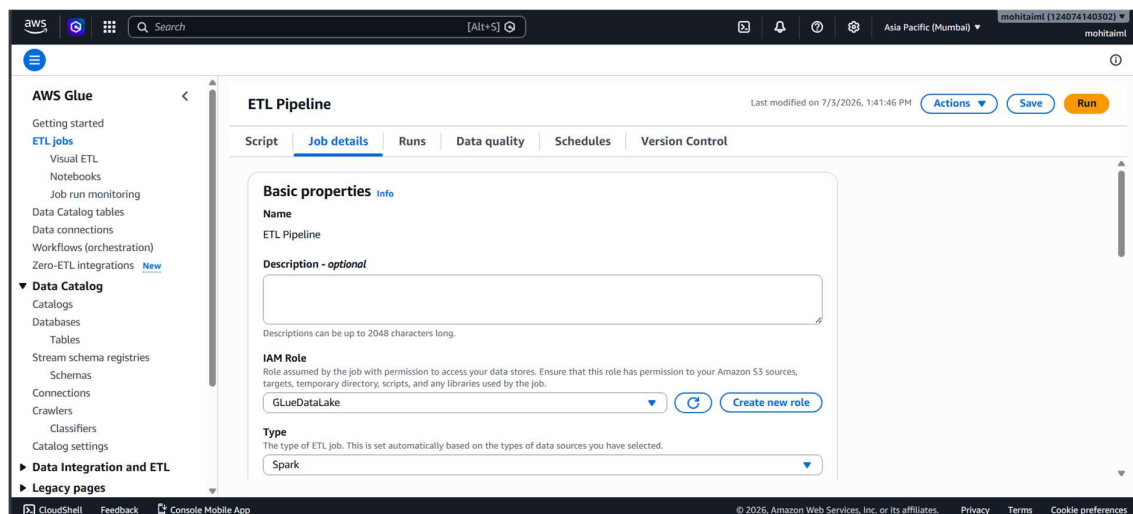
1. Navigate to Glue **ETL jobs** > **Spark script editor**.



2. Paste the complete script above into the editor workspace, adjusting the processed bucket name placeholder.



3. Assign the GlueDataLakeRole in the job settings properties.



4. Click **Save** and **Run**.

Verification Checkpoint (Step 3)

- Navigate to the **Job runs** monitoring page.
- Confirm the job status progresses to a green **Succeeded** notification.

The screenshot shows the AWS Glue console interface. On the left is a navigation menu with options like 'Getting started', 'ETL jobs', 'Data Catalog', and 'Data Integration and ETL'. The main panel displays the 'ETL Pipeline' details. The 'Runs' tab is active, showing a table of job runs. The first run is 'Succeeded' with 0 retries, starting at 07/03/2026 13:47:40 and ending at 07/03/2026 13:51:08, with a duration of 1 m 22 s. Below the table, the 'Run details' section shows input arguments, logs, run insights, metrics, troubleshooting analysis, and Spark UI.

- Navigate to your yourname-ecommerce-processed-zone S3 bucket via the console and confirm that three folders (orders/, customers/, products/) have been generated populated with compressed .parquet objects.

Step 4: Cataloging the Cleaned Data (AWS Glue Crawler)

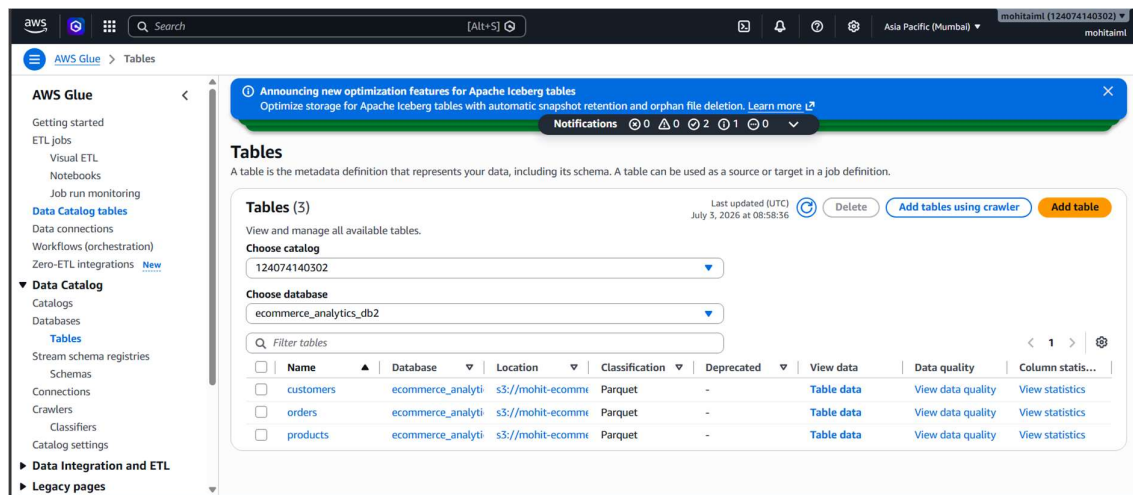
With your columnar Parquet data generated, a second crawler maps the optimized files into an analytics-ready presentation schema.

Implementation Steps

1. Navigate to **Crawlers** > **Create crawler**. Name it ecommerce-processed-crawler.
2. Add a new data source pointing specifically to the root of the processed zone: `s3://yourname-ecommerce-processed-zone/`.
3. Set the execution role to GlueDataLakeRole.
4. Create a separate target database named ecommerce_analytics_db.
5. Save and execute the crawler.

Verification Checkpoint (Step 4)

- Navigate to Glue **Tables** under the ecommerce_analytics_db target database.
- Verify that three distinct tables exist (orders, customers, products).
- Confirm their **Classification** column strictly indicates **parquet**.

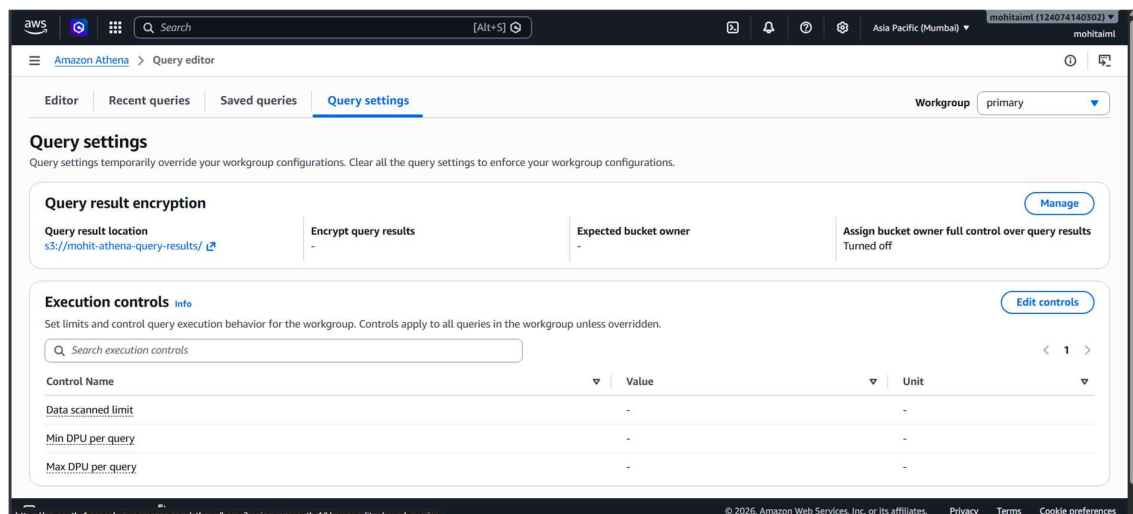


Step 5: Relational Analytics (Amazon Athena)

Amazon Athena acts as an ad-hoc serverless query engine, executing distributed SQL expressions directly over S3 objects using metadata mappings from the Glue Catalog.

Analytical Validation Pipeline

1. Open the **Amazon Athena Console**.
2. Select **Settings > Manage** and set a dedicated query scratchpad bucket (e.g., `s3://mohit-athena-query-results-unique/`) to preserve physical text logs.



3. Switch back to the **Editor** tab and set the Database context dropdown to `ecommerce_analytics_db`.
4. Run the validation join script below:

SQL

SELECT

o.order_id,

c.name AS customer_name,

c.email AS customer_email,

p.product_name,

p.category,

o.quantity,

p.price,

(o.quantity * p.price) AS total_sales

FROM orders o

INNER JOIN customers c ON o.customer_id = c.customer_id

INNER JOIN products p ON o.product_id = p.product_id

ORDER BY total_sales DESC;

The screenshot displays the Amazon Athena Query Editor interface. On the left, the 'Data' section shows the 'Data source' as 'AwsDataCatalog', 'Catalog' as 'None', and 'Database' as 'ecommerce_analytics_db2'. Below this, the 'Tables and views' section lists 'customers', 'orders', and 'products'. The main area shows 'Query 1' with a SQL query that selects various fields from the 'orders', 'customers', and 'products' tables, calculates 'total_sales', and orders the results by 'total_sales' in descending order. The query is as follows:

```
1 SELECT
2   o.order_id,
3   c.name AS customer_name,
4   c.email AS customer_email,
5   p.product_name,
6   p.category,
7   o.quantity,
8   p.price,
9   (o.quantity * p.price) AS total_sales
10  FROM orders o
11     INNER JOIN customers c ON o.customer_id = c.customer_id
12     INNER JOIN products p ON o.product_id = p.product_id
13  ORDER BY total_sales DESC;
```

At the bottom, there are buttons for 'Run again', 'Explain', 'Cancel', 'Clear', and 'Create'. A 'Reuse query results' option is also visible, indicating results are available up to 60 minutes ago.